BCSE307P Compiler Design Lab

Innovative Project

Compiler Design Playground: An Interactive Compiler Engineering Lab

by

Reg No	Team Member Names
[Reg. No. 1]	[Student Name 1]
[Reg. No. 2]	[Student Name 2]
[Reg. No. 3]	[Student Name 3]
[Reg. No. 4]	[Student Name 4]

Under the Supervision of

Prof. NIVITHA K

Assistant Professor Sr. Grade-2

School of Computer Science and Engineering



VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

November 2026

1. Abstract:

Compiler construction is normally taught as a sequence of separate algorithms – finite automata, parsing, attribute grammars, intermediate code, data-flow analysis and code generation – that students rarely see working together on one program. This project presents the **Compiler Design Playground**, a browser-based learning environment that connects these topics through a working compiler and makes every intermediate result visible.

The application is a client-side single-page application written in JavaScript with React 19 and Vite 8; all compiler logic runs in the browser in plain JavaScript modules, with no server. It has two parts. The **Compiler Studio** compiles programs written in MiniC, a documented subset of C with `int` and `bool` variables, arithmetic, comparisons, short-circuit logic, `if`/`else`, `while`, block scopes, `print` and `input()`, through seven connected phases: lexical analysis, recursive-descent parsing, semantic analysis, three-address-code generation, optimisation, register-allocating code generation, and execution on a virtual CPU that can be stepped instruction by instruction. The **Algorithm Lab** offers ten independent, interactive topics – from Thompson's construction and LL(1)/SLR(1) parsing to backpatching, data-flow analysis, graph-colouring register allocation and SSA form – each with visualisations and theory notes.

Correctness is checked rather than assumed: the optimised code and the CPU are compared against a reference interpreter on every compilation, and an automated suite of 234 tests, including 300 randomly generated programs, passes. The language deliberately omits functions, arrays and floating point, and register allocation in generated code is block-local.

2. Introduction:

2.1 Background

A compiler translates a program written in a high-level source language into an equivalent program in a target language, usually machine code. It does so in phases: a **lexical analyser** groups characters into tokens; a **parser** checks the tokens against a grammar and builds a syntax tree; **semantic analysis** resolves names and checks types; the front end then emits a machine-independent **intermediate representation (IR)**, which an **optimiser** improves before a **code generator** maps it onto the registers and instructions of a target machine. Compiler design matters well beyond writing compilers: the same techniques underlie interpreters, query engines, configuration languages, static analysers and the tooling every programmer uses daily.

2.2 Problem statement

In a typical course these phases are taught as isolated algorithms worked by hand on small textbook examples. Students compute FIRST and FOLLOW sets for one grammar, build an SLR table for another, and convert an expression to three-address code for a third, but rarely see one program flow through every phase, nor how the output of one phase becomes the input of the next. Hand calculation also gives no feedback when a step is wrong, and static diagrams cannot show dynamic behaviour such as a parser's stack or the effect of an optimisation on the executed program.

2.3 Motivation

An interactive environment can close this gap. If the student can type any program, compile it, and inspect the tokens, tree, symbol table, intermediate code, optimised code, machine instructions and machine state that result – and if the classic algorithms can also be explored individually on their own inputs – then theory and practice reinforce each other. Running entirely in the browser removes installation barriers, so the tool can be used in a lab session or at home with only a web browser.

2.4 Objectives

1. Define a small but coherent source language (MiniC) whose every feature is supported by every compiler phase.
2. Implement a complete pipeline – lexer, parser, semantic analyser, IR generator, optimiser, code generator and virtual CPU – that exchanges structured data between phases.

3. Let users inspect the real output of each phase, including errors, and step through program execution on the virtual CPU.
4. Provide independent laboratories for the standard algorithms of a compiler-design syllabus, with visualisations and theory notes.
5. Check the correctness of the optimiser and the code generator automatically, both during use and in an automated test suite.

2.5 Scope and intended users

The project is aimed at undergraduate students taking a compiler-design course and at instructors who want to demonstrate algorithms live. Its scope is educational: MiniC intentionally omits functions, arrays, strings and floating-point numbers so that every phase stays small enough to be read and understood, and the virtual CPU is a teaching machine rather than a real instruction-set architecture.

2.6 System overview

The application is organised in three layers (Figure 1 in Section 4). React components in `src/components/` handle presentation and interaction only. A pipeline layer in `src/pipeline/` defines the language and runs the phases through a single function, `compile(source, options)`. The compiler algorithms themselves live in `src/engine/` as plain JavaScript modules with no user-interface code, which allows them to be tested in isolation. The Compiler Studio calls `compile()` whenever the user compiles a program and renders the structured result of each phase; the Algorithm Lab calls the engines directly with its own inputs, or calls `compile()` without execution when it needs the IR of a MiniC program.

3. Novelty in Project:

The algorithms implemented in this project are standard textbook algorithms (Aho, Lam, Sethi and Ullman, and the papers cited in Section 6); the project does not claim to invent new compilation techniques. Its contribution is in how these algorithms are brought together and presented for learning. The following characteristics distinguish it from a collection of separate algorithm demonstrations.

1. One program through every phase

The Compiler Studio is a genuinely connected compiler: the AST produced by the parser is the input of the semantic analyser, whose symbol and type information drives IR generation, whose instructions are optimised, translated to target code and executed. Every panel shows the output

of the current program, not a pre-written example, and users can write, save, open and download their own programs.

2. Execution down to the machine level

Compilation does not stop at intermediate code. Generated instructions run on a virtual CPU that the user can step one instruction at a time while watching the program counter, registers, memory and output, with a trace that relates every target instruction to the TAC instruction it implements.

3. Correctness made visible

For every compilation the Studio runs the original and the optimised IR on the user's inputs and reports whether they behave identically, and compares the CPU's result with a reference IR interpreter. Students can therefore see not only what an optimisation does but evidence that it preserved the program's behaviour – while the interface states clearly that this is evidence for one input, not a proof.

4. Errors as part of the lesson

Lexical, syntax, semantic and run-time errors are reported with positions and clickable links into the source. The parser recovers from errors so several are reported at once; a failed phase marks all later phases as skipped while keeping earlier results visible, which shows how compilers stop and recover.

5. Independent laboratories alongside the compiler

The Algorithm Lab lets students explore algorithms that do not belong in a small compiler's main path – NFA-to-DFA conversion, LL(1) and SLR(1) table construction, operator-precedence parsing, backpatching, graph-colouring register allocation, SSA form, instruction scheduling and vectorisation analysis – on inputs of their choice, each with a visualisation and a theory panel. Topics 3–7 analyse IR produced by the same MiniC front end, linking the laboratories back to the compiler.

6. Zero installation

Everything runs in the browser; there is no server, database or external compilation service, so the tool can be used with only a web browser.

4. Proposed Work:

4.1 System architecture

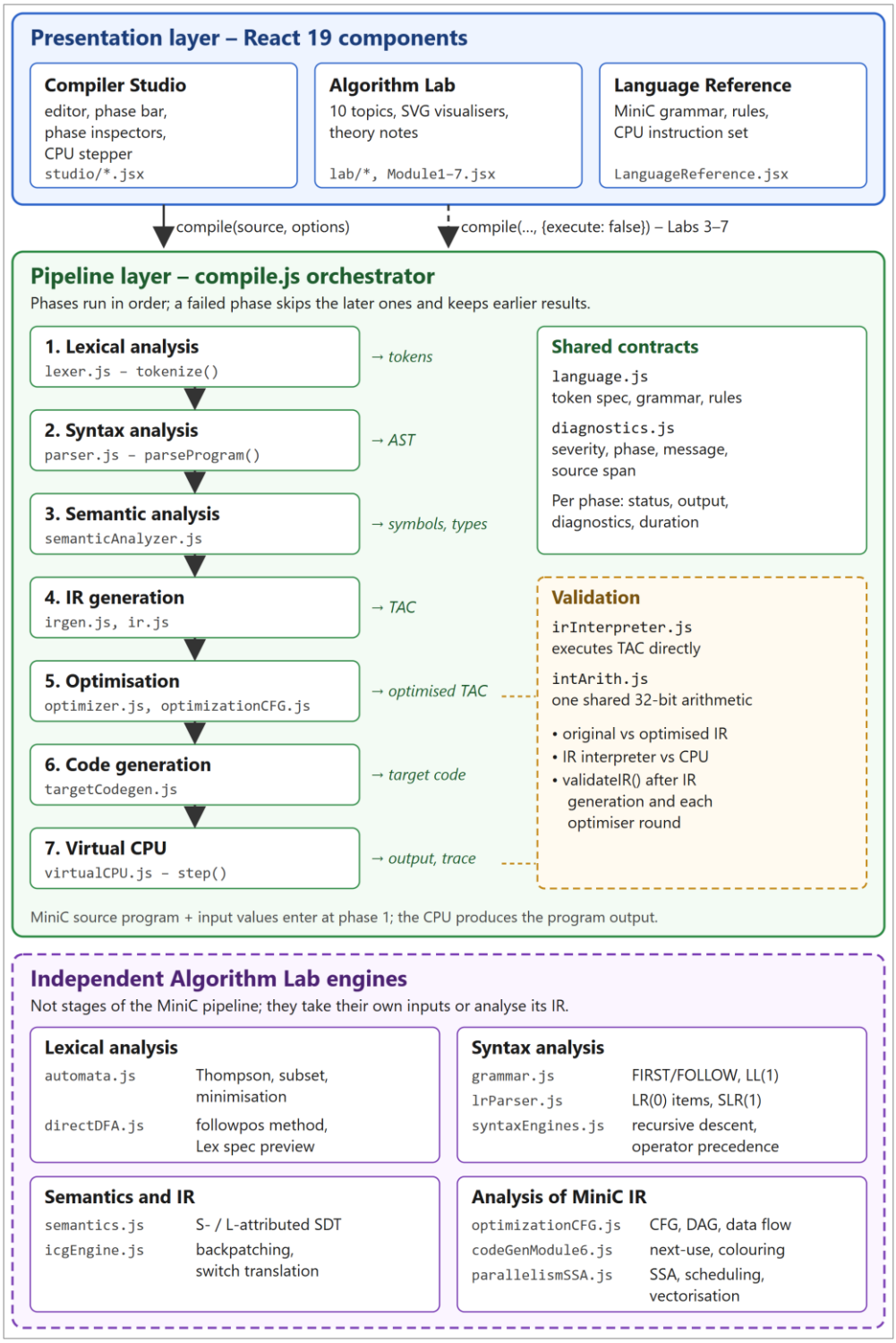


Figure 1. *Architecture of the Compiler Design Playground: presentation layer, pipeline layer with the seven compiler phases and their data, validation, and the independent Algorithm Lab engines.*

Figure 1 shows the architecture as implemented. The presentation layer contains the Compiler Studio, the Algorithm Lab and a Language Reference page. The pipeline layer coordinates the

seven compiler phases; the boxes below each phase name the data passed to the next phase. A reference interpreter validates the optimiser and the code generator. The Algorithm Lab engines at the bottom are independent of the pipeline: they are not compiler stages, although Lab topics 3–7 analyse IR produced by `compile()`.

4.2 Module responsibilities

Layer / module	Responsibility
`src/App.jsx`	Application shell; hash-based navigation between `#/studio`, `#/lab/<topic>` and `#/reference`.
`src/components/studio/`	Compiler Studio: source editor, program management, phase bar, one inspector per phase, CPU stepper.
`src/components/lab/`, `Module1–7`	Algorithm Lab shell, ten topic pages, theory notes, shared SVG visualisers (`Visualizer.jsx`).
`src/pipeline/compile.js`	Orchestrator: runs the phases in order, records status, output, diagnostics and timing per phase.
`src/pipeline/language.js`	MiniC definition: lexical specification, grammar, operator table, semantic rules.
`src/pipeline/diagnostics.js`	Common diagnostic format `{severity, phase, code, message, span}`.
`src/engine/*.js`	Compiler algorithms as pure functions (lexer, parser, semantic analyser, IR, optimiser, code generator, CPU, lab algorithms).

4.3 Source language: MiniC

MiniC was designed so that every construct it contains is supported by all seven phases. A program is a sequence of statements over `int` (32-bit, wrap-around) and `bool` variables. Constructs from C that MiniC does not support – `for`, functions, `return`, arrays, floating-point and string literals, `++` and compound assignment – are recognised and rejected with a specific diagnostic rather than a confusing syntax error. The grammar, taken from `src/pipeline/language.js`, is:

```
program        → statement* EOF
statement      → declaration | assignment | printStmt | ifStmt | whileStmt | block
declaration    → type IDENT ( "=" expression )? ";"
type           → "int" | "bool"
assignment     → IDENT "=" expression ";"
printStmt      → "print" "(" expression ")" ";"
ifStmt         → "if" "(" expression ")" body ( "else" body )?
whileStmt      → "while" "(" expression ")" body
block          → "{" statement* "}"
expression     → logicOr
logicOr        → logicAnd ( "||" logicAnd )*
logicAnd       → equality ( "&&" equality )*
equality       → relational ( ( "==" | "!=" ) relational )*
relational     → additive ( ( "<" | "<=" | ">" | ">=" ) additive )*
additive       → multiplicative ( ( "+" | "-" ) multiplicative )*
multiplicative → unary ( ( "*" | "/" | "%" ) unary )*
unary          → ( "-" | "!" ) unary | primary
primary        → INT_LITERAL | "true" | "false" | IDENT | "input" "(" ")" | "(" expression ")"
```

Division truncates toward zero and the remainder takes the sign of the dividend, as in C; division by zero is a run-time error. The observable behaviour of a program is its output and any run-time error.

4.4 The compilation pipeline

`compile(source, options)` executes the phases below in order. Each phase receives the structured output of its predecessor, never display text. If a phase reports an error, every later phase is marked *skipped* and the outputs of earlier phases are kept, so the user can still inspect them. An exception inside an engine is converted into an `internal-error` diagnostic instead of crashing the application.

Phase	Engine (function)	Input → output
1. Lexical analysis	`lexer.js` (`tokenize`)	source text → tokens with type, value, line, column and offsets
2. Syntax analysis	`parser.js` (`parseProgram`)	tokens → abstract syntax tree with node ids and source spans
3. Semantic analysis	`semanticAnalyzer.js` (`analyzeProgram`)	AST → symbol table, scopes, type of every expression, name resolutions
4. IR generation	`irgen.js` (`generateIR`), `ir.js`	typed AST → three-address code; quadruples, triples and CFG derived from it
5. Optimisation	`optimizer.js` (`optimizeIR`)	TAC → optimised TAC plus a log of every transformation
6. Code generation	`targetCodegen.js` (`generateTargetCode`)	TAC → instructions for the virtual CPU, with register decisions
7. Execution	`virtualCPU.js` (`assemble`, `step`, `runProgram`)	target program and inputs → output, final machine state, trace

Lexical analysis

The lexer applies an ordered list of token specifications (`MINIC_TOKEN_SPECS`): at each position the first matching pattern wins, so comments and multi-character operators such as `<=` and `&&` are listed before shorter patterns. Specifications may be marked *skip* (comments) or *error* (malformed lexemes such as `9abc` or an unterminated comment), so each malformed lexeme produces a single, positioned error.

Syntax analysis

A recursive-descent parser has one function per grammar rule. Each binary precedence level is written as a loop, which builds left-associative trees directly. On a syntax error the parser records a diagnostic and skips to the next statement boundary (panic-mode recovery), so several independent errors are reported in one run; every recovery step consumes input, which guarantees termination.

Semantic analysis

Names are resolved through a stack of scopes, one per block. The analyser reports undeclared variables and duplicate declarations, warns about shadowing and unused variables, and computes the type of every expression, checking operators, assignments and conditions against MiniC's typing rules. An erroneous sub-expression receives the type *error*, which later checks accept silently, so one mistake yields one message. Shadowed variables receive unique internal names (for example `x.2`) so that later phases can treat all variables uniformly.

Intermediate code generation

The typed AST is translated by syntax-directed translation into three-address code with eight instruction kinds: assignment, binary and unary operations, `input`, `print`, labels, unconditional jumps and conditional jumps of the form `if a relop b goto L`. Boolean conditions use jumping code, which implements the short-circuit semantics of `&&` and `||`; `if` and `while` branch on the negated test so that the true case falls through. Quadruples, triples and indirect triples are derived from the same instructions, and `validateIR` checks the structure, for example that every jump targets an existing label.

Optimisation

The optimiser repeats five passes until nothing changes (at most twelve rounds): global constant propagation and folding using an iterative data-flow analysis over the control-flow graph, local copy propagation, local common-subexpression elimination, liveness-based dead-code elimination, and control-flow clean-up (unreachable blocks, jumps to the next instruction,

unused labels). Each pass applies a transformation only when its precondition holds; instructions with side effects (`input`, `print`, branches) and divisions that might divide by zero are never removed. Constant folding uses the same 32-bit arithmetic module (`intArith.js`) as the interpreter and the CPU, so compile-time and run-time results cannot disagree.

Target code generation

The default code generator follows the simple code generator of Aho et al. (Section 8.6): for each basic block it maintains register descriptors (which names a register holds) and address descriptors (where the current value of each name is), reuses values already in registers, and, when all registers are busy, spills the register whose values are needed furthest in the future according to next-use information. Values that are live at the end of a block are stored to memory. A *naive* strategy that loads and stores around every instruction is provided for comparison, and the number of registers can be set between three and eight.

Virtual CPU

The virtual CPU is a load/store machine with 28 instructions (`LI`, `LD`, `ST`, `MOV`, arithmetic and set-on-comparison instructions, `NEG`, `NOT`, `IN`, `OUT`, `OUTB`, `JMP`, six conditional branches and `HALT`). Programs are assembled and validated before execution: every opcode, operand kind, register and label is checked. `step()` executes one instruction and returns a new state without modifying the old one, which makes reset and re-execution deterministic. Division by zero, reading an unwritten register or memory cell, running past the end of the program and exceeding the step budget (100 000 instructions by default) stop the CPU with a structured error.

4.5 Validation of the pipeline

A reference interpreter (`irInterpreter.js`) executes three-address code directly and defines the meaning of a program. On every compilation the Studio runs the original and the optimised IR on the user's inputs and compares their output and run-time errors; it also compares the virtual CPU with the interpreter. A mismatch would be reported as an error. These checks are evidence for the given input; broader evidence comes from the automated tests described in Section 5.8.

4.6 Independent algorithm laboratories

The Algorithm Lab groups ten topics by compiler phase. Each page combines an interactive tool, SVG visualisations and a collapsible theory panel (41 theory sections in total).

Lab topic	Algorithms implemented	Input
Regex → NFA → DFA	Thompson construction, ε-closure, subset construction, partition-refinement minimisation, string simulation	regular expression, test string
Direct DFA & tokenizer	nullable/firstpos/lastpos/followpos method, configurable tokenizer, Lex specification preview	regular expression, source text
Grammars & LL(1)	FIRST/FOLLOW, left-recursion elimination, left factoring, LL(1) table and predictive parse	grammar, token string
Parsing techniques	recursive descent, operator-precedence parsing, SLR(1) parse with trees	token string
LR(0) items & SLR(1)	canonical LR(0) collection, ACTION/GOTO table, conflict detection, shift-reduce trace	grammar, token string
Syntax-directed translation	S-attributed and L-attributed evaluation over real parse trees; MiniC type checking	arithmetic expression; MiniC program
TAC, quads, triples, backpatching	three-address code, quadruples, triples, indirect triples, one-pass backpatching, switch translation	MiniC program; boolean expression
Blocks, CFG, DAG, data flow	leaders and basic blocks, CFG, DAG of a block, reaching definitions, liveness, optimiser passes	MiniC program
Next-use, registers, frames	next-use information, interference graph and graph colouring, naive vs descriptor code, activation-record layout	MiniC program
SSA, scheduling, SIMD	dominators, dominance frontiers, pruned SSA with φ-functions, list scheduling, loop dependence analysis	MiniC program; array loop

4.7 User interaction

In the Compiler Studio the user selects an example or writes a program, enters values for `input()`, and compiles with a button or Ctrl+Enter. A phase bar shows the status of each of the seven phases (success, warning, error, skipped or disabled); selecting a phase opens its inspector. Compiler options switch individual optimisation passes on or off, choose the code-generation strategy and the number of registers, and can make final variable values observable. User programs are kept in the browser's local storage and can be opened from or downloaded to `.mc` files.

5. Implementation & Screenshots:

5.1 Technologies and development environment

Item	Used in the project
Language	JavaScript (ES modules), JSX
User interface	React 19.2 with React DOM; icons from lucide-react
Build tool and dev server	Vite 8 with `@vitejs/plugin-react`
Styling and graphics	Hand-written CSS (`index.css`, `layout.css`, `theme.css`); visualisations drawn as SVG in React
Testing	Node.js built-in test runner (`node:test`, `node:assert`); no additional test libraries
Linting	oxlint
Runtime	Any modern web browser; Node.js is required only for development
Size (approx.)	engines ≈ 6 000 lines, pipeline ≈ 550, user interface ≈ 6 600, tests ≈ 1 700 lines

5.2 Application structure and navigation

`App.jsx` implements a small hash router, so the browser's back button and shareable links work without a routing library. The header offers three sections – Compiler Studio, Algorithm Lab and Language Reference – and the Lab has a sidebar listing its ten topics grouped by compiler phase. Figure 2 shows the main interface with the Optimization showcase example compiled.

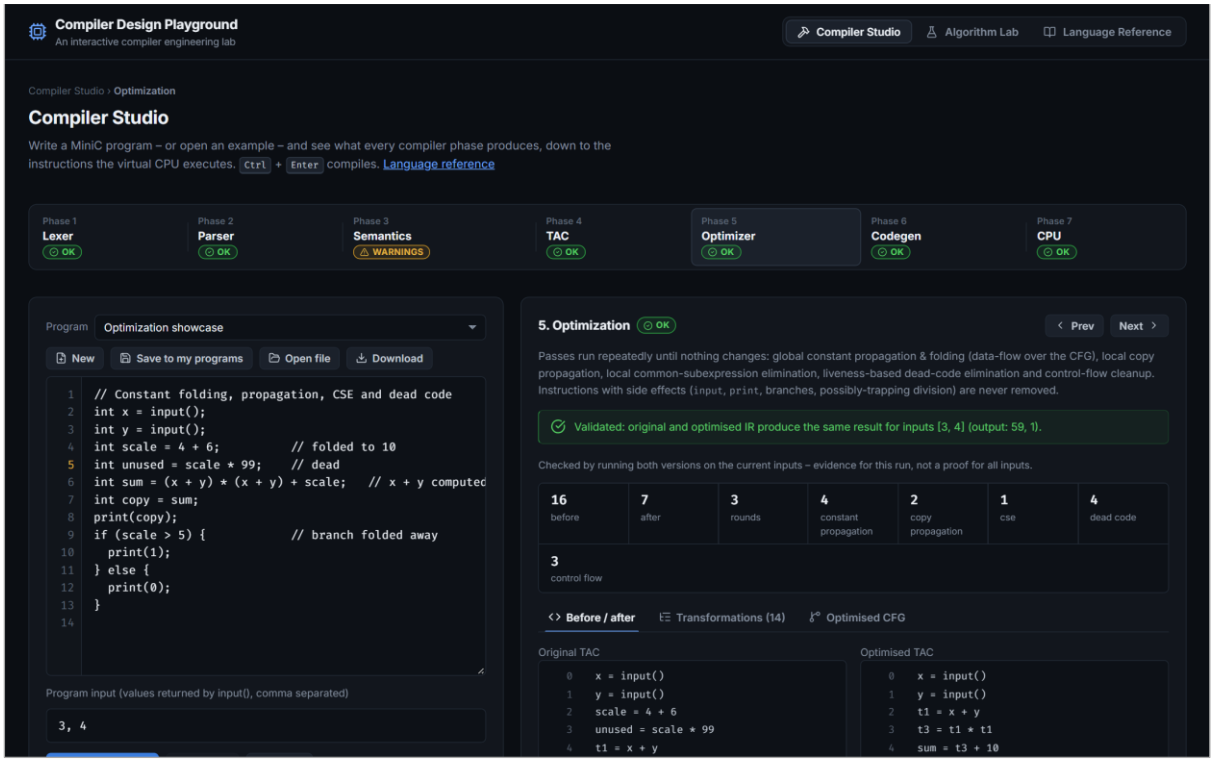


Figure 2. *Main interface of the Compiler Design Playground: the Compiler Studio with the seven-phase status bar, the source editor and the optimisation inspector.*

5.3 Data flow between the user interface and the engines

When the user compiles, `CompilerStudio.jsx` calls `compile(source, options)` and stores the returned object. The object contains one entry per phase with its `status`, `output`, `diagnostics` and duration, plus the combined diagnostics and the phase at which compilation stopped. Each inspector component (`PhaseViews.jsx`, `CPUView.jsx`) renders only this structured output; display text such as `t1 = x + y` is produced by formatting functions (`formatInstruction`, `formatTargetInstruction`) from the data and is never parsed back. The CPU inspector receives the assembled program and calls `step()` to advance one instruction at a time.

5.4 Front end: lexical, syntax and semantic analysis

The lexical inspector lists every token with its type and line:column position; clicking a token selects the lexeme in the editor (Figure 3). The syntax inspector draws the AST, offers an outline view and shows the grammar (Figure 4). The semantic inspector shows the symbol table with internal names, types, scopes and read/write counts, and an AST annotated with the type of every expression (Figure 5).

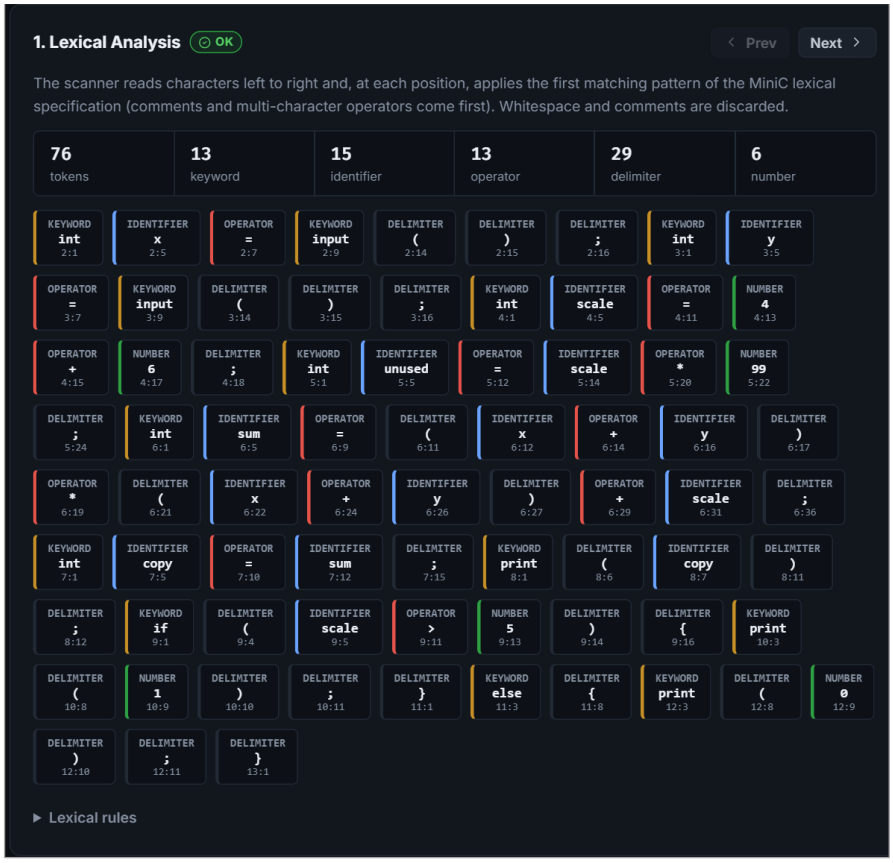


Figure 3. *Lexical analysis: token stream with token types and source positions for the Optimization showcase program.*

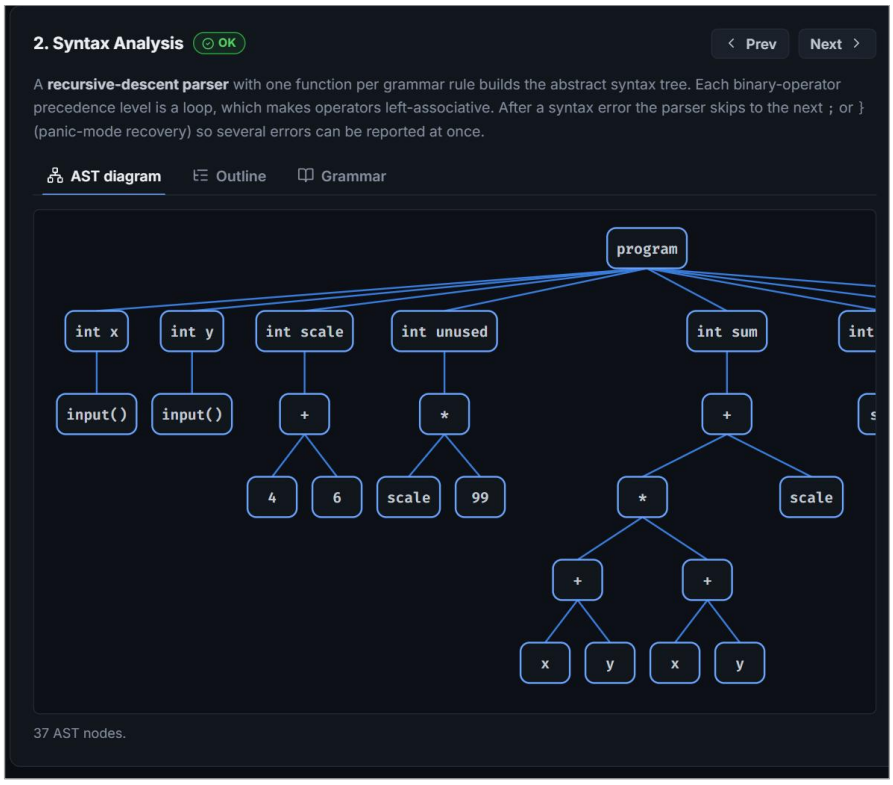


Figure 4. *Syntax analysis: abstract syntax tree produced by the recursive-descent parser (the diagram scrolls horizontally in the application; the left part is shown).*

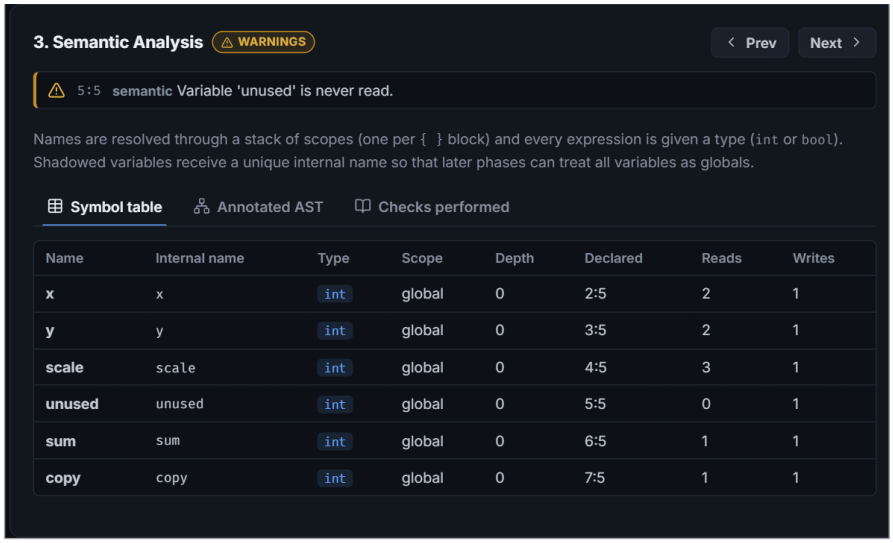


Figure 5. *Semantic analysis: symbol table with internal names, types, scopes and reference counts.*

5.5 Intermediate code, optimisation, code generation and execution

The IR inspector shows the three-address code with its quadruple, triple and indirect-triple forms and the basic blocks and control-flow graph (Figures 6 and 7). In the example, the optimiser reduces 16 instructions to 7 in three rounds: it folds `4 + 6` to `10`, propagates the constant into later uses, reuses the common subexpression `x + y`, removes the unused variable and folds the constant branch (Figure 8). The code-generation inspector annotates each group of target instructions with the TAC instruction it implements (Figure 9). Running the program on the virtual CPU with inputs 3 and 4 executes 10 instructions and prints 59 and 1, matching the reference interpreter (Figure 10).

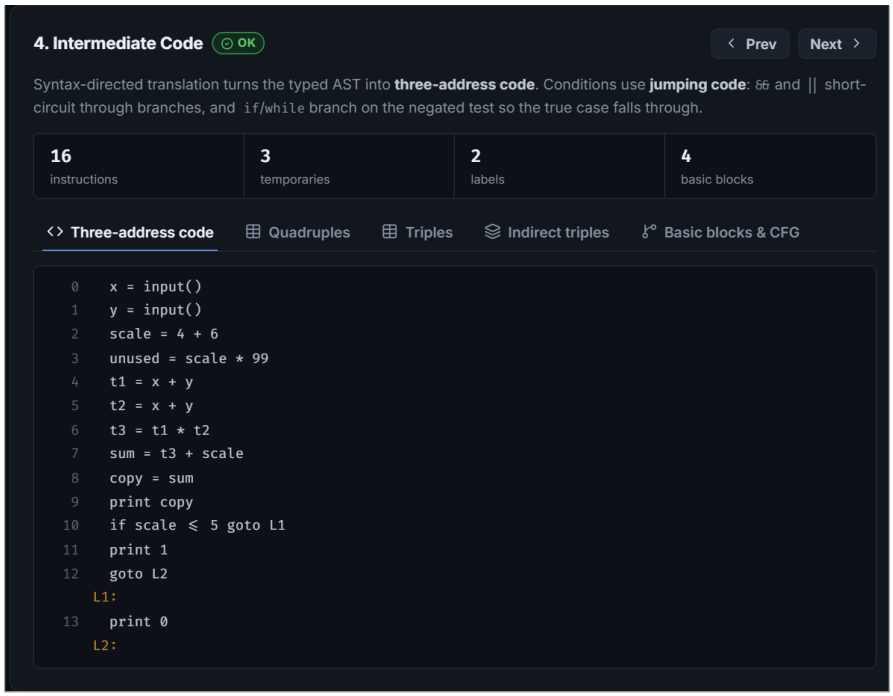


Figure 6. *Intermediate code: three-address code generated from the typed AST.*

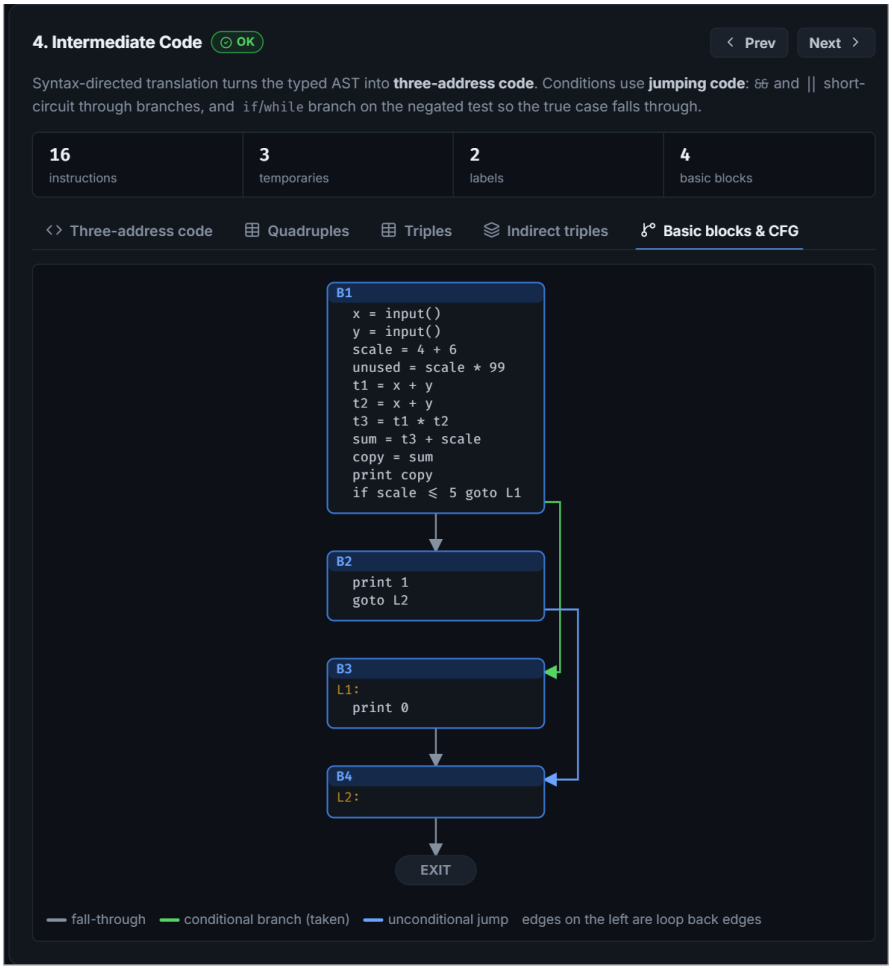


Figure 7. *Basic blocks and control-flow graph of the unoptimised three-address code (green: conditional branch taken; blue: unconditional jump).*

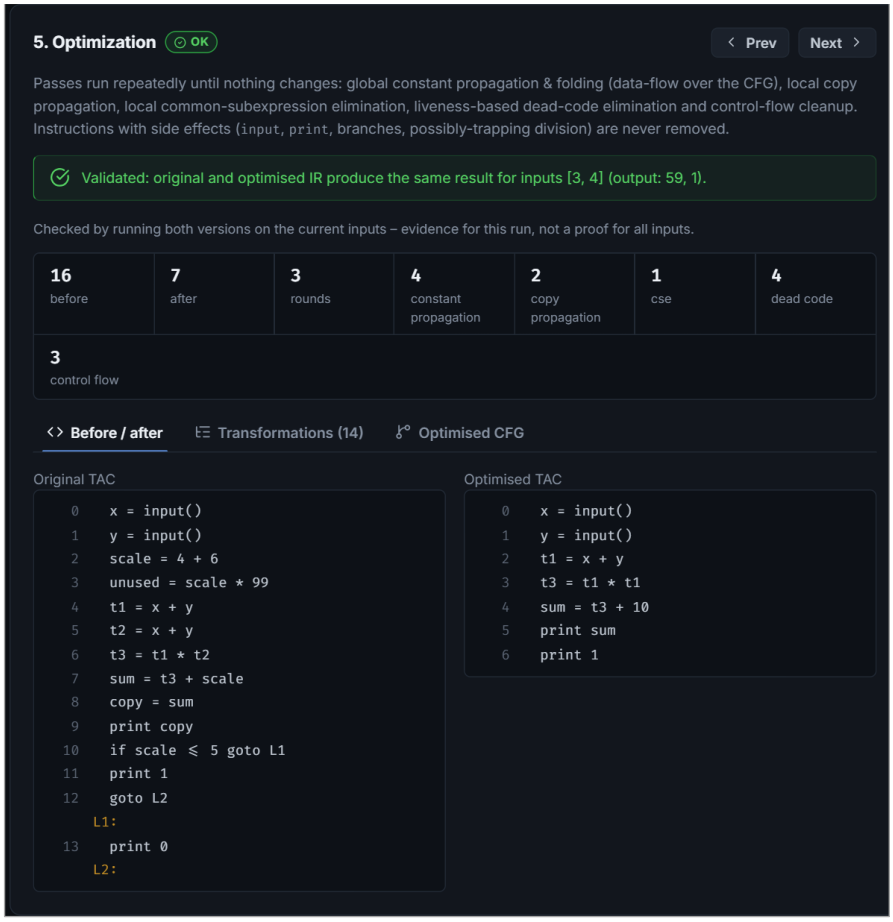


Figure 8. *Optimisation: validation banner, per-pass statistics, and original versus optimised three-address code.*

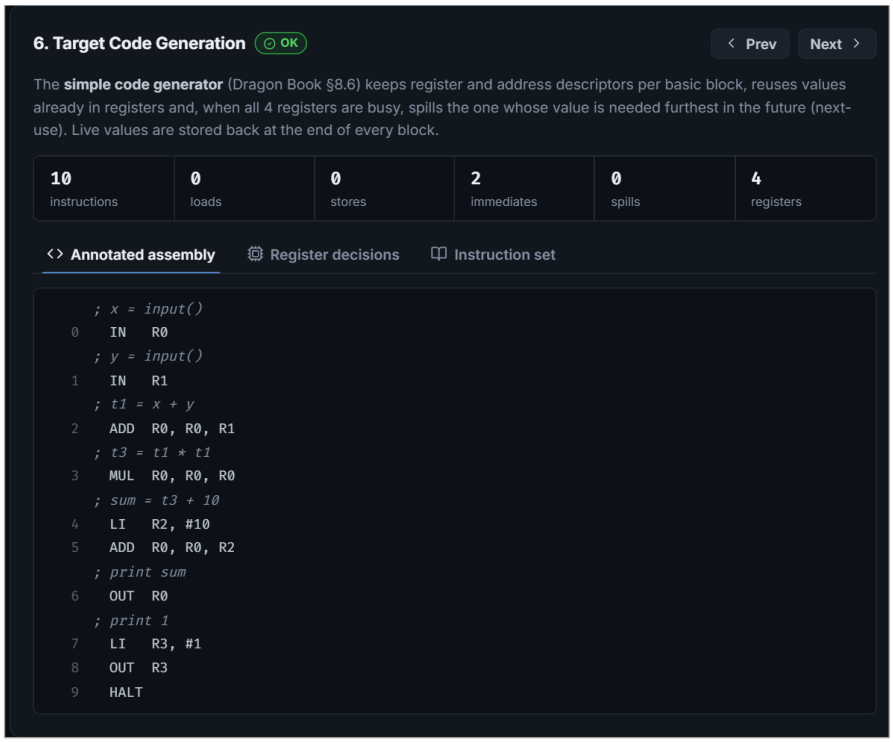


Figure 9. *Target code generation: virtual-CPU assembly annotated with the source TAC instructions, with load, store and spill statistics.*

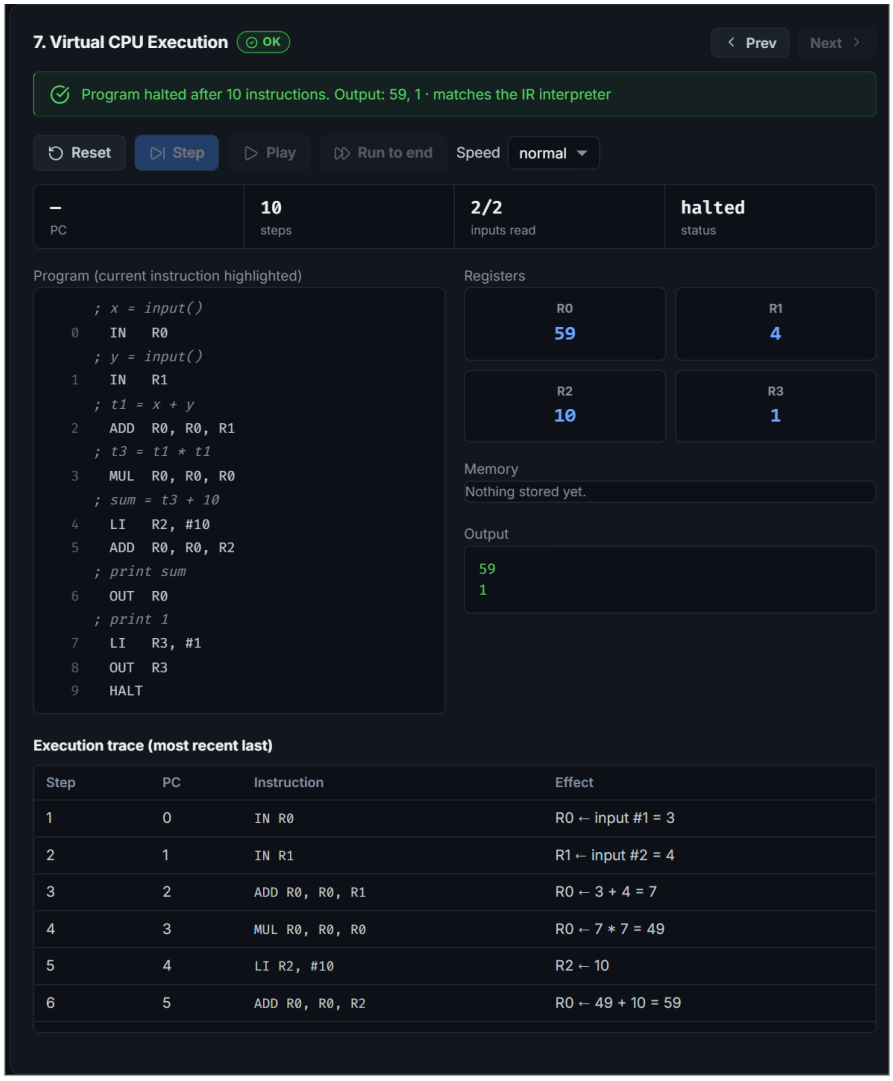


Figure 10. *Virtual CPU after execution: program with current instruction, registers, memory, output and execution trace.*

5.6 Input validation and error handling

Every phase reports problems as diagnostics with a severity, a phase, a code, a message and a source span. The editor marks error lines in its gutter, and clicking a diagnostic selects the offending text. Figure 11 shows the Syntax errors example: the parser reports two errors in one run thanks to panic-mode recovery, all later phases are marked skipped, and the partial AST of the statements that parsed is still displayed. Other safeguards include validation of the program-input field (integers within 32-bit range), the CPU's step budget against infinite loops, and structured run-time errors such as division by zero.

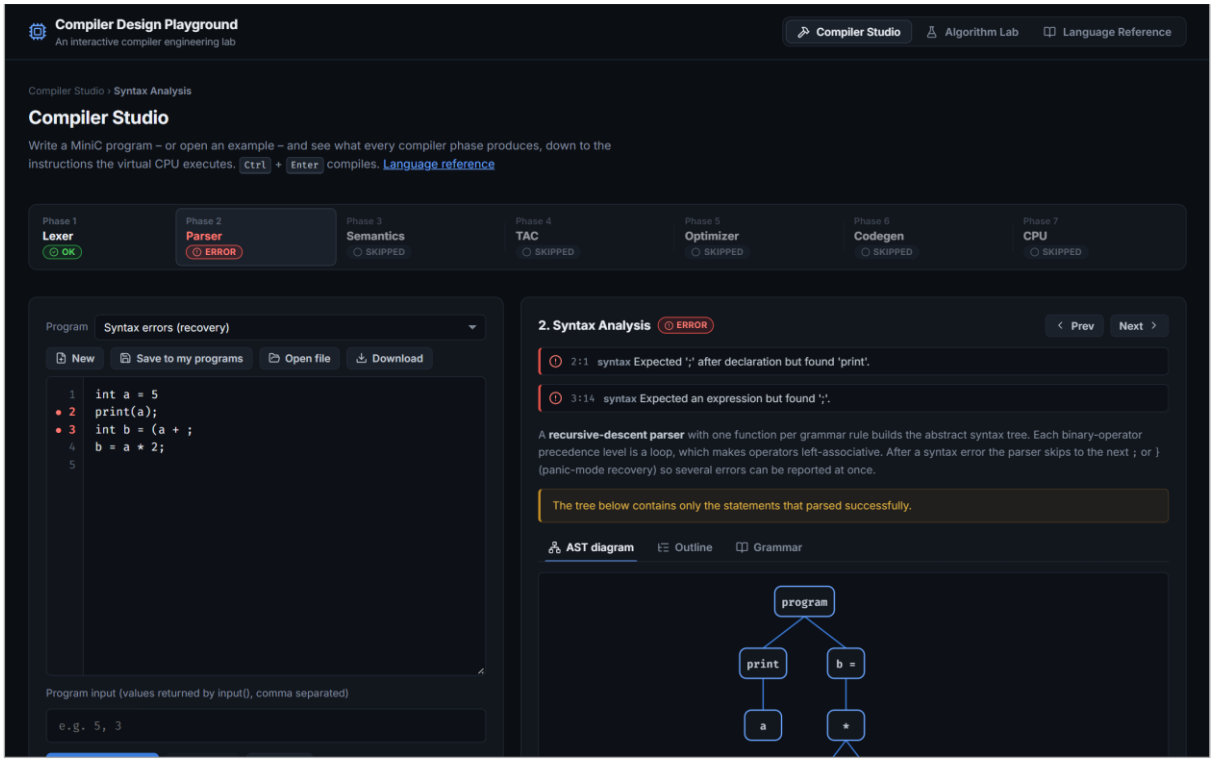


Figure 11. *Error handling: two syntax errors reported with positions; dependent phases are skipped while lexical results and the partial AST remain available.*

5.7 Algorithm Lab

Figures 12 to 20 show representative laboratories. The automata laboratory reduces the five-state subset-construction DFA for `(a|b)*abb` to the textbook minimal DFA with four states and lists each partitioning step (Figure 12). The grammar laboratory builds a conflict-free LL(1) table and traces the predictive parse of `id + id * id` (Figure 13). The LR laboratory shows the SLR(1) ACTION/GOTO table with conflict detection (Figure 14). The SDT laboratory evaluates `3 + 5 * 2` to 13 on an annotated parse tree (Figure 15). The backpatching laboratory reproduces the Dragon Book example `a < b || c < d && e < f`, leaving truelist {100, 104} and falselist {103, 105} open (Figure 16). Further laboratories show the DAG of a basic block (Figure 17), graph-colouring register allocation (Figure 18) and SSA form with φ-functions for the GCD program (Figure 19). Every topic has a theory panel (Figure 20).

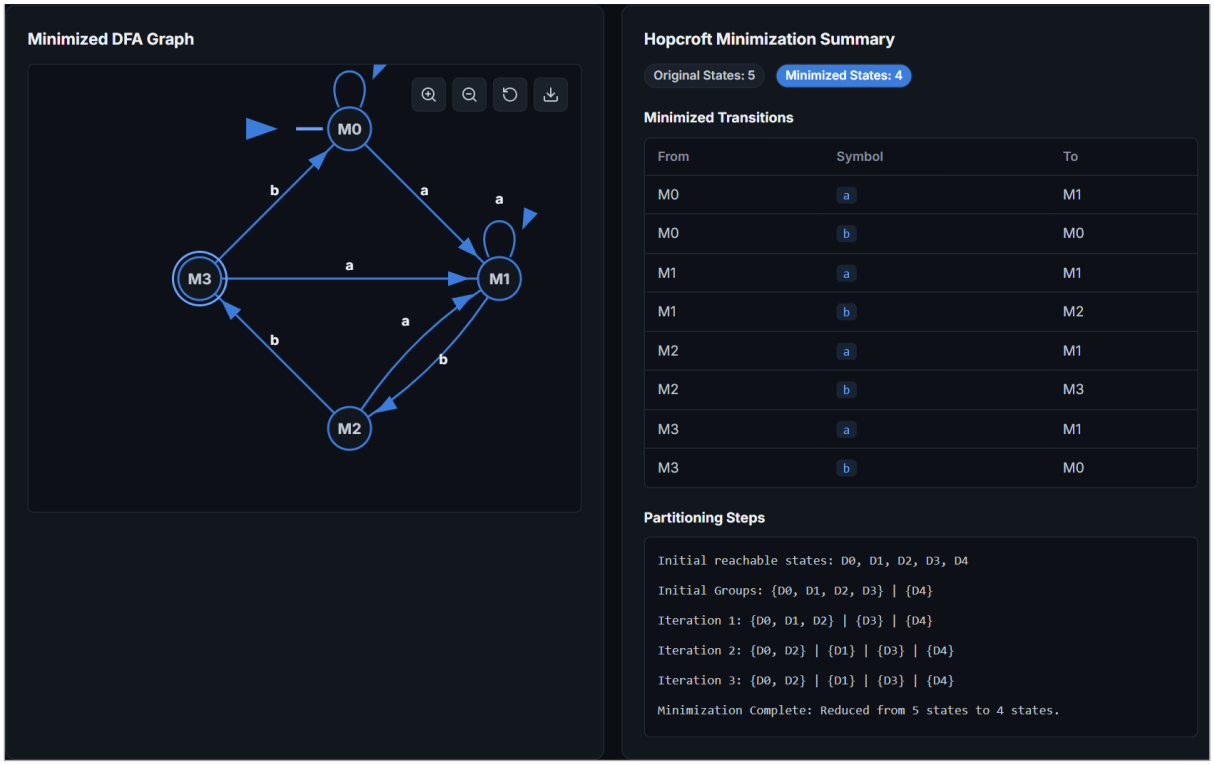


Figure 12. *Algorithm Lab – automata: minimised DFA for (a|b)*abb with the partition-refinement steps.*

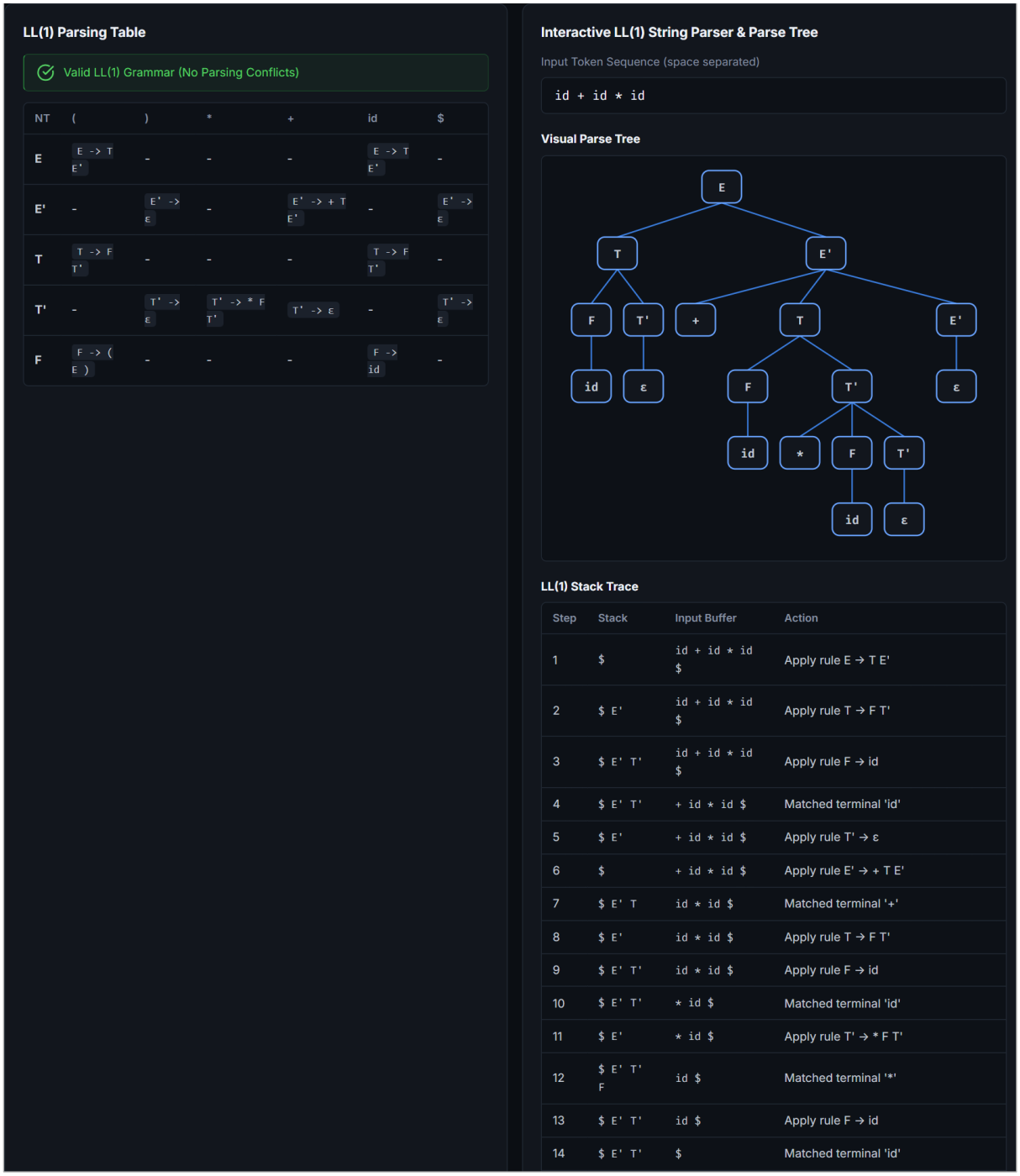


Figure 13. *Algorithm Lab – grammars: LL(1) parsing table, parse tree and stack trace for id + id * id.*

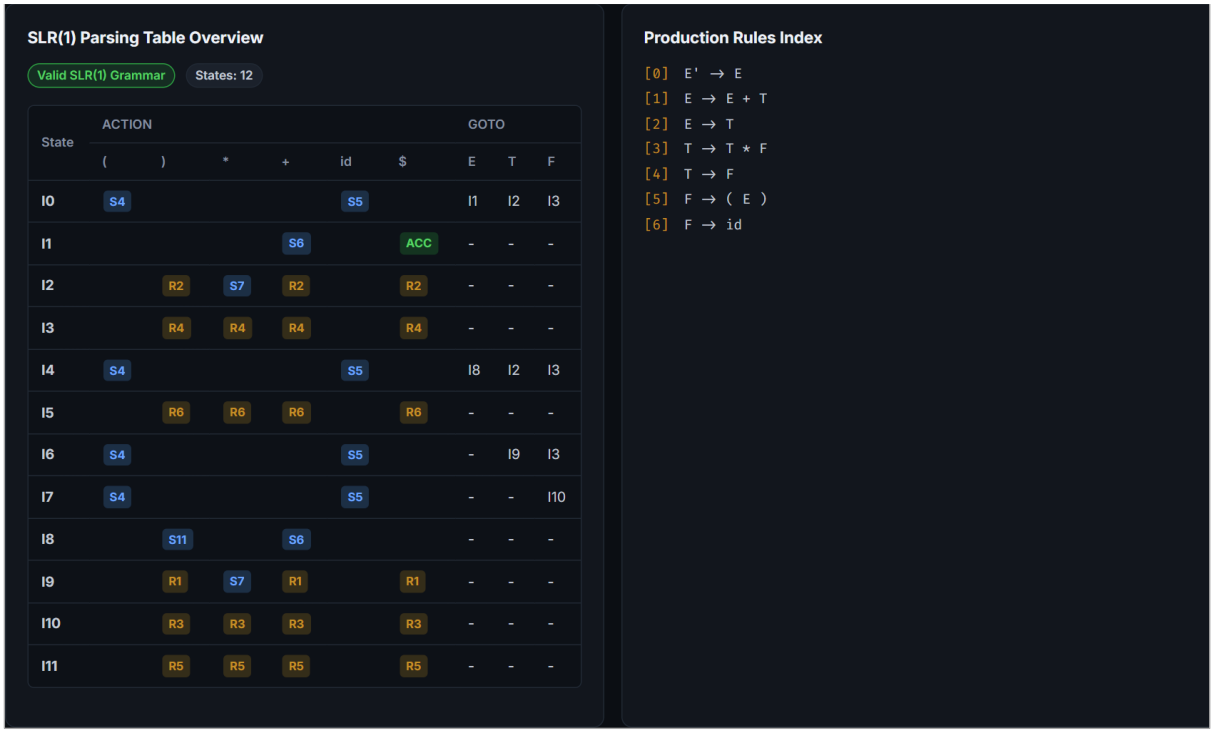


Figure 14. *Algorithm Lab – LR parsing: SLR(1) ACTION/GOTO table for the expression grammar and the production index.*

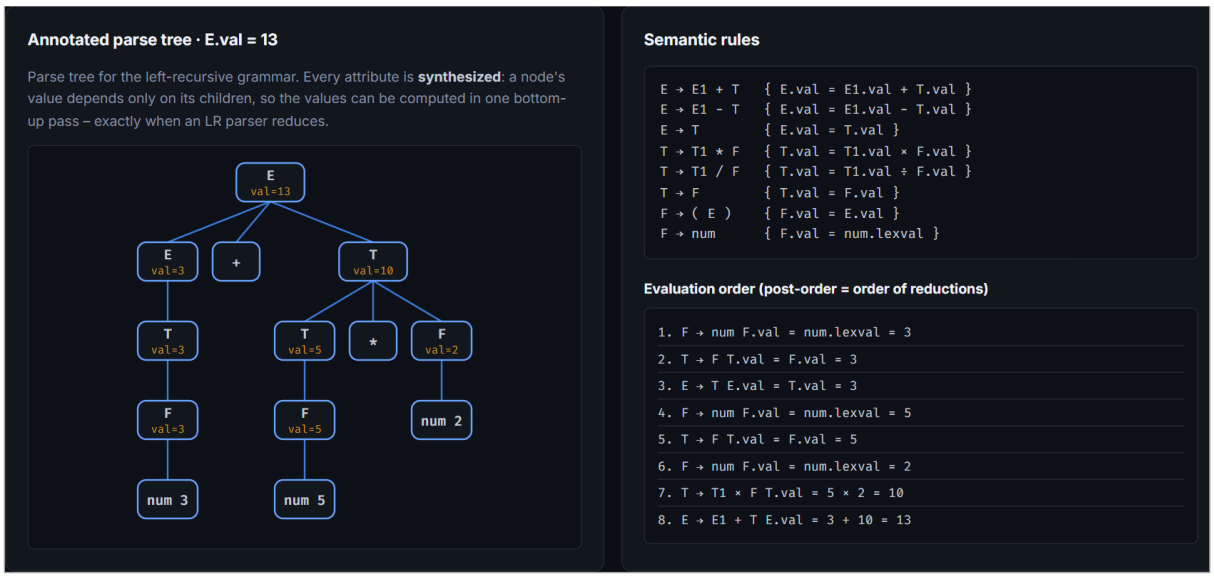


Figure 15. *Algorithm Lab – syntax-directed translation: S-attributed annotated parse tree and evaluation order for 3 + 5 * 2.*

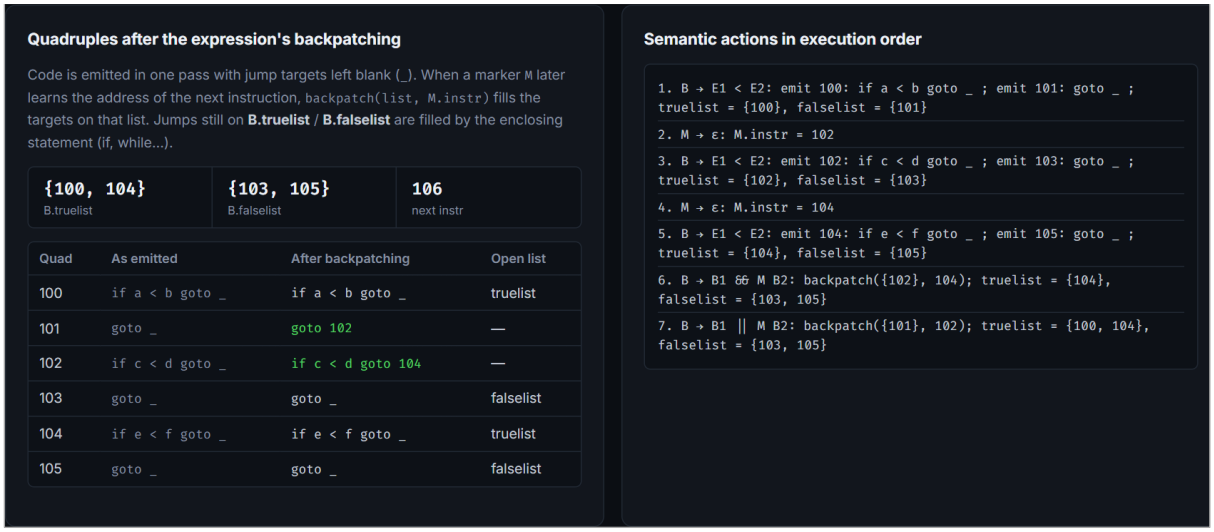


Figure 16. *Algorithm Lab – backpatching of a < b || c < d && e < f, with the semantic actions in execution order.*

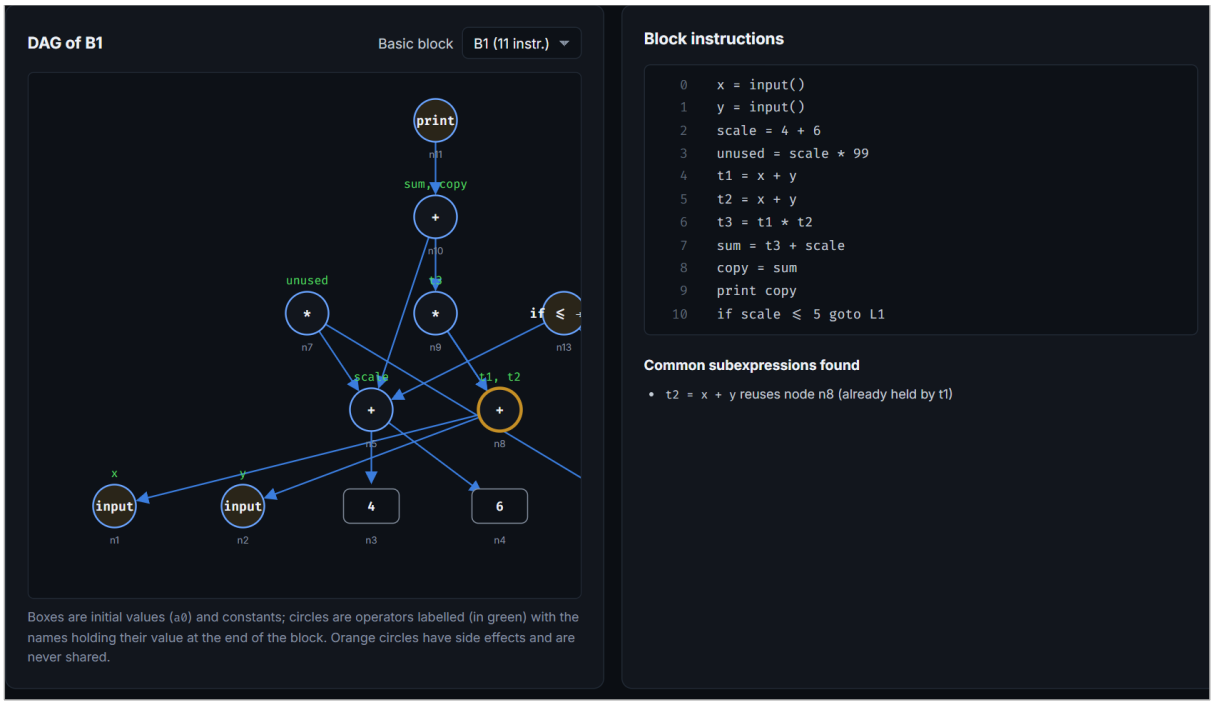


Figure 17. *Algorithm Lab – optimisation: DAG of a basic block with the detected common subexpression.*

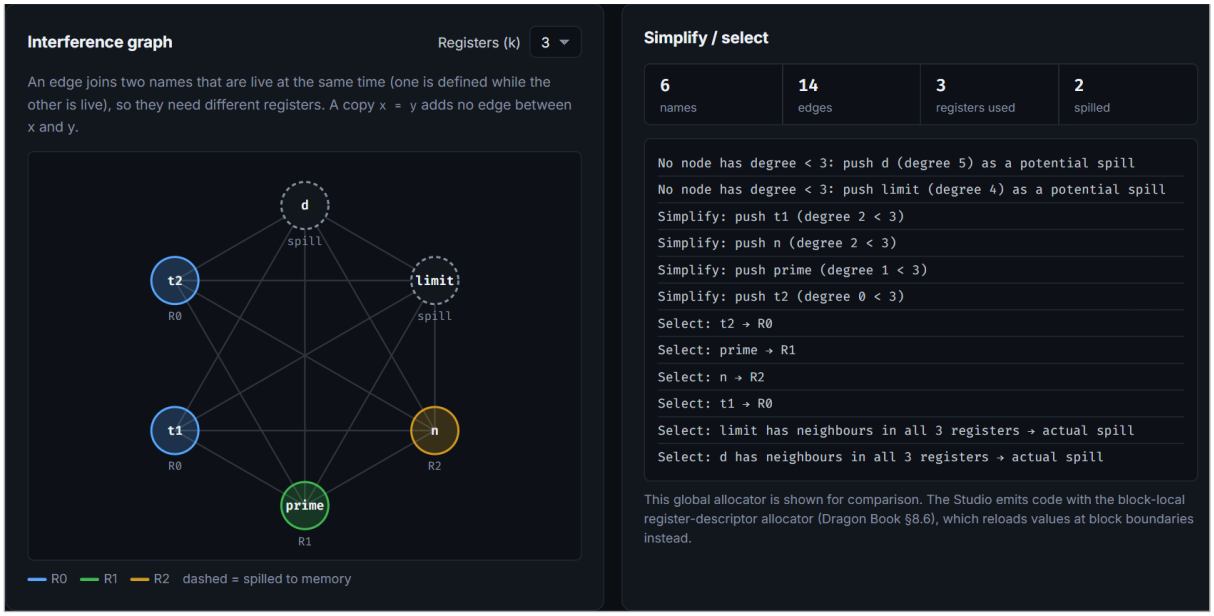


Figure 18. *Algorithm Lab – code generation: interference graph coloured with three registers and the simplify/select log.*

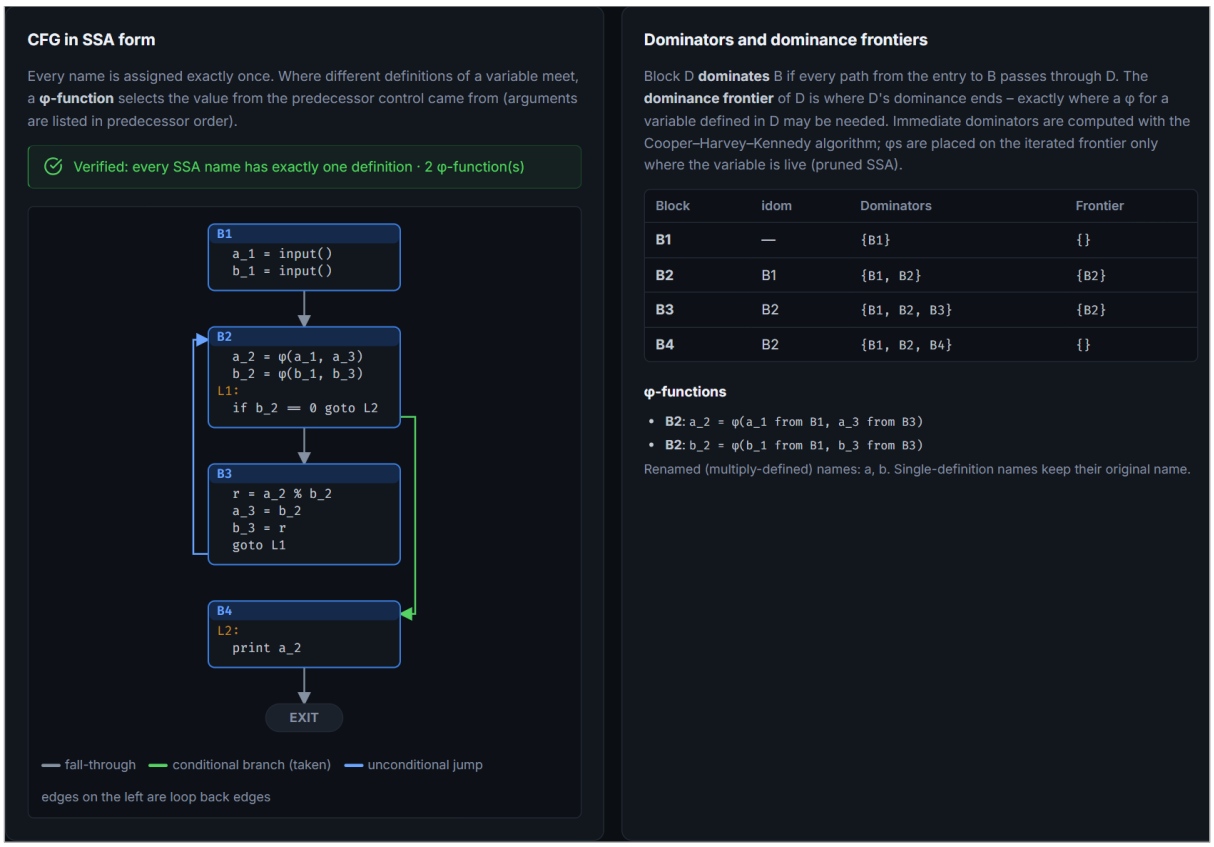


Figure 19. *Algorithm Lab – SSA form of the GCD program with φ-functions, dominators and dominance frontiers.*

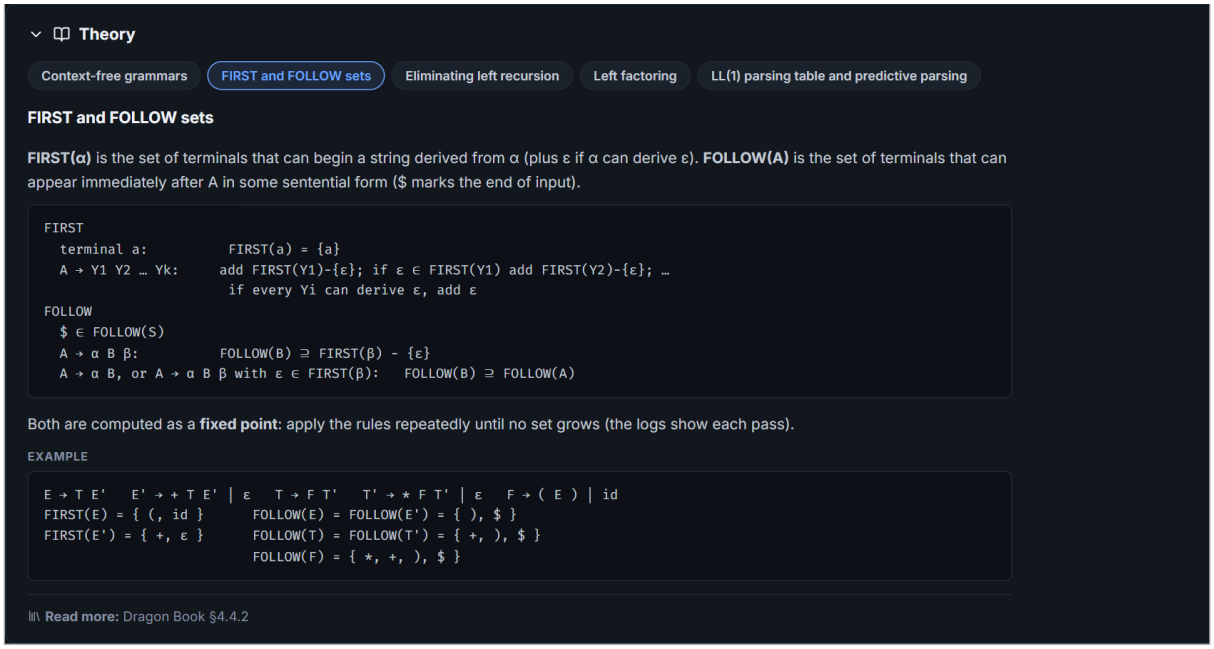


Figure 20. *Theory panel of the Grammars & LL(1) topic (FIRST and FOLLOW sets).*

5.8 Testing

The engines and the pipeline are covered by an automated test suite run with `npm test`. On 26 October 2026 all 234 tests in 12 files passed. Expected program outputs were calculated by hand, not by running the compiler, and automata are checked against JavaScript's regular-expression engine on every string up to length six.

Test file	Tests	What is checked
`automata.test.js`	16	Thompson NFA, subset construction, minimisation and direct DFA against JavaScript RegExp; textbook followpos table
`grammar.test.js`, `syntaxLab.test.js`	17	FIRST/FOLLOW, LL(1), left recursion and factoring, SLR(1), operator precedence, recursive descent
`frontend.test.js`	26	MiniC lexer, parser (precedence, associativity, recovery) and semantic analysis
`ir.test.js`	18	IR generation, 32-bit arithmetic, reference interpreter, quadruples and triples
`optimizer.test.js`	65	each pass, 24-program corpus, equivalence of every pass alone and combined, CFG and data flow
`cpu.test.js`	35	instruction semantics and faults; CPU against the interpreter for every corpus program with 3, 4 and 6 registers
`fuzz.test.js`	1	300 randomly generated programs: interpreter, optimised IR and CPU must agree
`pipeline.test.js`	28	end-to-end scenarios, error propagation between phases, all bundled examples

Test file	Tests	What is checked
`labEngines.test.js`, `advanced.test.js`	26	SDT, backpatching, switch, SSA, scheduling, vectorisation, next-use, graph colouring
`theory.test.js`	2	every Lab topic has well-formed theory notes

To confirm that the tests can detect real mistakes, bugs were deliberately inserted into the copy-propagation, common-subexpression, dead-code and constant-propagation passes and into the register-spilling code; the test suite failed for each of them.

5.9 Running the project locally

1. Install a current Node.js release (Vite 8 requires Node.js 20.19+ or 22.12+; the project was developed with Node.js 24).
2. In the project folder, install the dependencies with `npm install`.
3. Start the development server with `npm run dev` and open the address it prints (by default http://localhost:5173).
4. Run the automated tests with `npm test`; create a production build in `dist/` with `npm run build`.

6. Conclusion:

The Compiler Design Playground implements a complete, connected compiler for MiniC, a documented subset of C, together with ten independent laboratories covering the standard algorithms of a compiler-design course. A program typed by the user passes through lexical, syntax and semantic analysis, three-address-code generation, a five-pass optimiser and a register-allocating code generator, and is executed on a virtual CPU that can be stepped instruction by instruction. Each phase's real output – tokens, syntax tree, symbol table, intermediate code in several notations, control-flow graph, optimisation log, annotated assembly and machine state – can be inspected, and errors are reported with positions at the phase where they occur.

For learning, the project links algorithms that are usually studied separately: students can see how a grammar becomes a parser, how a tree becomes intermediate code, what an optimisation changes, and how registers and memory evolve as the generated code runs. The laboratories let them experiment with automata, grammars, parsing tables, backpatching, data-flow analysis, register allocation and SSA on their own inputs, supported by theory notes.

The most significant demonstrated capabilities are the end-to-end compilation and execution of MiniC programs, the per-compilation check that optimised code and generated machine code behave like the original program, and an automated test suite of 234 passing tests including differential testing on 300 random programs.

6.1 Limitations

- MiniC has no functions, arrays, strings, floating-point numbers or `for` loops.
- Register allocation in generated code is local to basic blocks; the graph-colouring allocator is shown in the Lab for comparison but is not used to emit code.
- Copy propagation and common-subexpression elimination are local to basic blocks, and there is no loop-invariant code motion.
- Only SLR(1) is implemented among the LR methods; canonical LR(1) and LALR(1) are described but not implemented.
- The activation-record view describes the layout the variables would have on a stack; the virtual CPU addresses memory by name.
- The equivalence checks shown in the Studio validate the current input only; they are evidence, not a proof of correctness.

6.2 Future work

- Add functions with a real run-time stack and calling convention, and arrays.
- Use a global register allocator in the code generator and add loop optimisations such as loop-invariant code motion.
- Implement LALR(1) parsing table construction in the LR laboratory.
- Allow MiniC programs to be compiled to SSA form within the main pipeline and optimised there.

6.3 References

- A. V. Aho, M. S. Lam, R. Sethi and J. D. Ullman, Compilers: Principles, Techniques, and Tools, 2nd ed., Addison-Wesley, 2006.
- K. D. Cooper, T. J. Harvey and K. Kennedy, “A Simple, Fast Dominance Algorithm,” Software Practice and Experience, 2001.
- R. Cytron, J. Ferrante, B. K. Rosen, M. N. Wegman and F. K. Zadeck, “Efficiently Computing Static Single Assignment Form and the Control Dependence Graph,” ACM TOPLAS, vol. 13, no. 4, 1991.
- G. J. Chaitin, “Register Allocation and Spilling via Graph Coloring,” Proc. SIGPLAN Symposium on Compiler Construction, 1982.
- R. Allen and K. Kennedy, Optimizing Compilers for Modern Architectures, Morgan Kaufmann, 2001.

7. Project Repository Link: (GitHub or drive link):

https://github.com/Kalpesh2276/Compiler-Design-Playground

The repository contains the source code (`src/`), the automated tests (`tests/`) and documentation (`README.md`, `docs/ARCHITECTURE.md`, `docs/LANGUAGE.md`, `docs/TESTING.md`, `docs/DEMO.md`).